Posibilidades teniendo en cuenta restricciones

Antes de estudiar las posibles restricciones vamos a calcular los posibles emprajamientos de los equipos.

Consideramos un emparejamiento como un lista de 20 elementos(porque tenemos 20 equipos) que se emparejan dos a dos. El primer partido tiene 20 posibles lugares, el segundo 19 y así sucesivamente. Por tanto tenemos realmente las permutaciones de estos 20 elementos, es decir, $20!$. Estos son todos los posibles emparejamientos teniendo en cuenta el orden de los partidos. Por tanto, estamos considerando como distintos emparejamientos todos los posibles ordenes para los mismos partidos. Es decir, que para los mismos 10 partidos cada orden diferente es un emparejamiento diferente. Por el planteamiento que vamos a utilizar queremos que esto no sea así ya que los distintos ordenes los vamos a considerar en los horarios. Para ello dividimos entre $10!$ para no considerar el orden.

Obtenemos entonces que todos los posibles emparejamientos distintos para nuestro 20 equipos son $\frac{20!}{10!}$.

Ahora vamos a considerar las restricciones. Tenemos 12 posibles franjas horarias repartidas en 4 días de la siguiente manera: 1 el Viernes y Lunes y 5 el Sábado y Domingo. Además todos los días se tiene que jugar un partido y los partidos pueden coincidir en una misma franja horaria. Aplicaremos para resolver esta parte el Principio de Inclusión-Exclusión.

Primero vamos a definir ciertas propiedades y conjuntos:

V: no se juegue el Viernes, S: no se juegue el Sábado, D: no se juegue el Domingo, L: no se juegue el Lunes.

S_0: Total de posibilidades sin restricciones, S_1: Posibilidades que no se juegue un día, S_2: Posibilidades que no se juegue dos días, S_3: Posibilidades que no se juegue tres días, S_4: Posibilidades que no se juegue cuatro días.

De esta forma los posibles horarios válidos en función de las restricciones, llamaremos N a este conjunto, vienen dadas por la siguiente expresión según el Principio de Inclusión-Exclusión:

$N = S_0 - S_1 + S_2 - S_3 + S_4$

Vamos a calcular el cardinal de los conjuntos por separado.

S_0: Claramente dado que los partidos pueden coincidir las posibilidades totales sin restricción son 12^{10}

S_1: Tenemos dos opciones.

Que se dé L o V: En este caso las posibilidades para cada situación son 11^{10}

Que se dé S o D: En este caso las posibilidades para cada situación son 7^{10}

Por tanto, $S_1 = 2 \cdot 11^{10} + 2 \cdot 7^{10}$

S_2: Tenemos varías opciones.

$V \cap L$: Posibildades son 10^{10}

$V \cap S, V \cap D, S \cap L, D \cap L$: Posibildades son 6^{10}

$S \cap D$: Posibildades son 2^{10}

Por tanto, $S_2 = 10^{10} + 4 \cdot 6^{10} + 2^{10}$

S_3: Tenemos varías opciones.

$V \cap S \cap D, L \cap S \cap D$: Posibildades son 1^{10}

$V \cap L \cap S, V \cap L \cap D$: Posibildades son 5^{10}

Por tanto, $S_3 = 2 \cdot 1^{10} + 2 \cdot 5^{10}$

S_4: Es imposible. 0 posibilidades.

Entonces, tenemos que:

$$N = 12^{10} - (2 \cdot 11^{10} + 2 \cdot 7^{10}) + 10^{10} + 4 \cdot 6^{10} + 2^{10} - (2 \cdot 1^{10} + 2 \cdot 5^{10}) + 0$$

Por último, una vez consideradas las restricciones, tenemos que las posibilidades con restricción para nuestro problema son:

$\frac{20!}{10!} \cdot N$

Posibilidades sin tener en cuenta restricciones

Para el problema sin restricciones los posibles emparejamientos son los mismos, es decir, $\frac{20!}{10!}$.

La diferencia viene en los posibles horarios. Como no tenemos restricciones, no hace falta que todos los días se juegue. Además, los partidos de igual manera se pueden solapar. El número de posibilidades en este caso ya lo hemos calculado antes. Se corresponde con S_0.

Entonces, las posibilidades sin tener en cuenta las restricciones son:

$\frac{20!}{10!} \cdot 12^{10}$

Estructura de datos que mejor se adapta al problema.

La estructura de datos que mejor se adapta al problema es la lista de python y sus "variantes"(diccionarios y tuplas). Como comentamos en el código del algoritmo genético, un diccionario no deja de ser una lista de pares clave-valor y una tupla una lista inmutable.
Las categorías de los equipos los definimos en una tupla, las audiencias bases(Sábado 22:00h), los coeficientes de reducción y los coeficientes por concurrencia en un diccionario, los horarios en una lista y el cromosoma de cada jornada en una lista de tuplas de tres elementos. Al final estamos todo el rato usando estos tipos de datos. Por ello además utilizamos tantos bucles for para recorrer las variables.

Solución mediante algoritmo genético

In [ ]:
```
from __future__ import annotations # Para trabajar con anotaciones. Nos permiten
# variables, parámetros de funciones y valores de retorno. Lo vimos en clase en

from typing import List, Dict, Tuple # Para trabajar con tuplas(equipos_cat), li

# Antes vamos a recordar lo que es cada variable

# Listas: coleccion ordenada y mutable de elementos, que puede contener datos de

# Tuplas: es una lista pero inmutable.

# Diccionarios: es una lista pero que almacena pares clave-valor. El tipico ejem
# persona = {"nombre": "Ana", "edad": 25, "ciudad": "Madrid"}


import random # Para hacer todo lo que tenga que ver con números aleatorios. Gen



# 1. DATOS DEL PROBLEMA


# Lo primero que vamos a hacer es definir todas las constantes y funciones que r


# Equipos y Categorías

# Definimos una tupla que define la categoria de cada equipo segun las audiencia
EQUIPOS_CAT = (['A'] * 3 + ['B'] * 7 + ['C'] * 10)
# Esto nos devolvería  ['A', 'A', 'A', 'B', 'B', 'B', 'B', 'B', 'B', 'B', 'C', '



# Define un diccionario que almacena la audiencia base para un partido jugado el
AUDIENCIA_BASE_SAB_22H: Dict[Tuple[str, str], float] = {
    ('A', 'A'): 5.0, ('A', 'B'): 4.3, ('A', 'C'): 3.5,
    ('B', 'B'): 3.5, ('B', 'A'): 4.0,  ('B','C'): 2.0,
    ('C', 'C'): 1.0, ('C', 'A'): 2.8, ('C', 'B'): 1.7,

}


# Definimos una funcion que dados los indices
```

```python
def get_audiencia_base(eq1_idx: int, eq2_idx: int) -> float: # Toma dos enteros(
    # Obtenemos las categorias usando el indice
    cat1 = EQUIPOS_CAT[eq1_idx]
    cat2 = EQUIPOS_CAT[eq2_idx]
    # Obtenemos la clave del diccionario
    key = tuple((cat1, cat2))
    # Devolvemos la audiencia del partido
    return AUDIENCIA_BASE_SAB_22H.get(key, 0.0)


# Horarios y Coeficientes de penalizacion y solapamiento

# Definimos una lista de los 12 nombres de las franjas horarias disponibles.
# Utilizaremos los índices numéricos (0 a 11) para representar los horarios en e
HORARIOS = [
    "Vie 20:00", "Lun 20:00",
    "Sab 12:00", "Sab 16:00", "Sab 18:00", "Sab 20:00", "Sab 22:00",
    "Dom 12:00", "Dom 16:00", "Dom 18:00", "Dom 20:00", "Dom 22:00"
]

# Definimos un diccionario con los coeficientes de audiencia de cada partido(tom
COEF_HORARIO = {
    "Vie 20:00": 0.8, "Lun 20:00": 0.7,
    "Sab 12:00": 0.6, "Sab 16:00": 0.7, "Sab 18:00": 0.8, "Sab 20:00": 0.9, "Sab
    "Dom 12:00": 0.5, "Dom 16:00": 0.6, "Dom 18:00": 0.7, "Dom 20:00": 0.8, "Dom
}
# Definimos el numero de franjas horarias posibles de los partidos
NUM_HORARIOS = len(HORARIOS)



# Definimos un diccionario con los coeficientes de reduccion de las audiencias s
COEF_CONCURRENCIA: Dict[int, float] = {
    0: 1.0, 1: 0.75, 2: 0.6, 3: 0.5,
    4: 0.4, 5: 0.35, 6: 0.3, 7: 0.25,
}
# Observemos que hemos hecho una transformacion. Por ejemplo para 2 coincidencia
# Es simplemente dos formas de trabajar con el mismo concepto aritmeticamente. D
# coeficiente.

# Parámetros del Algoritmo Genetico. Definimos el tamaño de la poblacion, numero
# Utilizamos una penalizacion tan grande que se resta a la audiencia para descar
# Por ejemplo que un equipo juegue dos veces.
POPULATION_SIZE = 100
NUM_GENERATIONS = 5000
PENALTY_VIOLATION = 1_000_000.0


# 2. FUNCION FITNESS



# Definimos una funcion que nos calcula el numero de coincidencias en cada franj
#  Recorre el cromosoma y genera un diccionario donde la clave es el horario y e
def calcular_concurrencia(cromosoma: List[Tuple[int, int, int]]) -> Dict[int, in
    # devuelve un diccionario con el conteo de partidos por horario.

    concurrencia: Dict[int, int] = {h_idx: 0 for h_idx in range(NUM_HORARIOS)} #
    # (desde 0 hasta NUM_HORARIOS-1) tiene un contador inicializado a cero.
    for _, _, h_idx in cromosoma: # Itera sobre cada tupla en el cromosoma, extr
```

```
        concurrencia[h_idx] += 1 # Incrementamos el contador de concurrencia en
    return concurrencia



# Ahora definimos la funcion fitness que es la encargada de determinar que tan b
# En este caso para ver como de buena es una solucion tenemos que calcular su au
def fitness(cromosoma: List[Tuple[int, int, int]]) -> float: # Toma un cromosoma

    # Verificar que todos los dias se juega y que un equipo solo juega una vez

    equipos_jugando = set() # Definimos un conjunto vacio para los equipos
    dias_con_partido = set() # Lo mismo para los dias
    invalido = False # Para validar la jornada. Tienen que cumplir las restricci


    for local, visitante, horario_idx in cromosoma:
        if local in equipos_jugando or visitante in equipos_jugando or local ==
            invalido = True # Si se cumple alguna condicion invalidamos la jorna
            break
        # Vamos añadiendo los equipos al conjunto de los que estan jugando segun
        equipos_jugando.add(local)
        equipos_jugando.add(visitante)

        # Identificamos los dias con los indices para luego hacer la comprobacio
        if horario_idx in (0, 1): dias_con_partido.add("VL") # Asocia los índice
        elif horario_idx >= 2 and horario_idx <= 6: dias_con_partido.add("Sab")
        elif horario_idx >= 7: dias_con_partido.add("Dom") # Asocia los índices

    if len(equipos_jugando) != 20 or invalido:
        return -PENALTY_VIOLATION # Si no estan jugando todos o si alguno juega

    # Ahora comprobamos los dias de juego
    if len(dias_con_partido) < 3: # si no se juegan todos los dias aplicamos tam
         return -PENALTY_VIOLATION

    # Una vez comprobadas las restricciones calculamos la audiencia total

    # Primero vemos si hay solapamiento con la funcion que definimos antes
    concurrencia_horarios = calcular_concurrencia(cromosoma)
    puntuacion_audiencia = 0.0 # Inicializamos la audiencia en 0


    for local, visitante, horario_idx in cromosoma:  # Recorremos el cromosoma p
        base = get_audiencia_base(local, visitante) # Obtenemos la audiencia bas
        horario_str = HORARIOS[horario_idx] # Obtenemos la franja horaria del pa
        coef_h = COEF_HORARIO[horario_str] # Obtenemos el coeficiente de audienc
        num_coincidentes = concurrencia_horarios[horario_idx] # Vemos si hay coi
        coef_c = COEF_CONCURRENCIA.get(num_coincidentes, COEF_CONCURRENCIA[7]) #
        # Usamos el máximo de reducción si hay mas de 7 coincidencias

        # Hacemos el calculo de la audiencia del partido
        audiencia_partido = base * coef_h * coef_c
        # Calculamos la audiencia de la jornada
        puntuacion_audiencia += audiencia_partido

    # Devolvemos audiencia segun solapamientos y horarios.
    return puntuacion_audiencia

# Observemos que como tenemos varios return la funcion ejecutara el primero que
```

```python
# Definimos una funcion para expresar el cromosoma del individuo(jornada) que es
# Usaremos la funcion para devolver la mejor solucion encontrada una vez aplicad
def formatear_jornada(cromosoma: List[Tuple[int, int, int]]) -> str: # Toma el c
    lineas = ["Solución de Jornada Óptima:"]
    for local, visitante, horario_idx in cromosoma: # Recorremos cada partido de
        local_cat = EQUIPOS_CAT[local] # Obtenemos categoria del local segun ina
        visitante_cat = EQUIPOS_CAT[visitante] # Obtenemos categoria del visitan
        lineas.append(f"  {HORARIOS[horario_idx]}: Equipo {local+1} ({local_cat}
    return "\n".join(lineas) # Une todas las líneas de la lista en una sola cade




# 3. ALGORITMO GENETICO


# Ahora vamos a definir el algoritmo genetico como tal. Utiliza la funcion fitne
# Esta parte es muy parecida a los codigos de los ejemplos que vimos en clase.

# Como vimos en los ejemplos de clase utilizamos programacion orientada objetos.

class GeneticAlgorithm:

    # Definimos la funcion constructor para inicializar los atributos del algori
    def __init__(self,
        population_size: int = 100,
        elitism_rate: float = 0.3,
        crossover_rate: float = 0.8,
        mutation_rate: float = 0.7,
        num_generations: int = 5000,
        seed: int = None):
        # Definimos el tamaño de poblacion, tasa de elitismo, tasa de mutacion,
        self.population_size = population_size
        self.elitism_rate = elitism_rate
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.num_generations = num_generations
        if seed is not None:
            random.seed(seed)

        # Definimos el numero de equipos y de partidos y los indices de los hora
        self.num_teams = 20
        self.num_matches = 10
        self.allowed_horarios = list(range(len(HORARIOS)))


    # Definimos una funcion para generar individuos(jornadas) aleatorias.
    def random_individual(self) -> List[Tuple[int, int, int]]: # Nos devuelve un
        teams = list(range(self.num_teams)) # Crea una lista del 0 al 19
        random.shuffle(teams) # la desordena al azar para emparejar equipos
        jornada = [] #Inicializamos la jornada que contendra 10 partidos
        for i in range(0, self.num_teams, 2): # Tomamos los equipos de dos en do
            local = teams[i]
            visitante = teams[i+1]
            horario = random.choice(self.allowed_horarios)
            jornada.append((local, visitante, horario))
        return jornada
```

```
    # Funcion para generar la poblacion a partir de la funcion que genera indivi
    def init_population(self) -> List[List[Tuple[int, int, int]]]:
        return [self.random_individual() for _ in range(self.population_size)] #
        #Es un bucle for pero escrito en una linea. Lo vimos en clase.

    # Funcion para evaluar la poblacion
    def evaluate(self, population: List[List[Tuple[int, int, int]]]) -> List[Tup
        puntuados = [(fitness(ind), ind) for ind in population] # Crea parejas (
        puntuados.sort(key=lambda x: x[0], reverse=True) # Ordena la población a
        return puntuados

    # Funcion para elegir el padre
    def tournament_select(self, scored_pop: List[Tuple[float, List[Tuple[int, in
        contenders = random.sample(scored_pop, k) # Elige k (3) individuos al az
        winner = max(contenders, key=lambda x: x[0])[1] # El que tiene el fitnes
        return winner[:]

    # Funcion para el cruce genetico, cambia dos jornadas para generar hijos
    def one_point_crossover_jornada(self, a: List[Tuple[int, int, int]], b: List
        if len(a) != len(b):
            return a[:], b[:]
        point = random.randint(1, len(a) - 1) #  Elige un partido al azar entre
        hijo1 = a[:point] + b[point:] # El hijo1 hereda los primeros partidos de
        hijo2 = b[:point] + a[point:]
        return hijo1, hijo2

    # Funcion para la mutacion de padres a hijos
    def mutate(self, ind: List[Tuple[int, int, int]]) -> None:
        if random.random() < self.mutation_rate:  # Con un 70% de probabilidad,
            match_idx = random.randrange(len(ind))
            local, visitante, current_h = ind[match_idx]
            new_h = random.choice([h for h in self.allowed_horarios if h != curr
            ind[match_idx] = (local, visitante, new_h)

        if random.random() < self.mutation_rate / 2.0: # Con una probabilidad me
            match_idx = random.randrange(len(ind))
            local, visitante, h = ind[match_idx]
            ind[match_idx] = (visitante, local, h)

    # Funcion para la evolucion de la poblacion
    def evolve(self):
        # Creamos la población inicial y la evalúamos por primera vez
        population = self.init_population()
        scored = self.evaluate(population)
        # Registramos cuál es la mejor jornada encontrada al inicio.
        best_score, best_ind = scored[0]
        # Calculamos cuántos individuos de la poblacion se salvarán.
        elite_count = max(1, int(self.elitism_rate * self.population_size))


        for gen in range(self.num_generations):
            new_population: List[List[Tuple[int, int, int]]] = [ind[:] for _, in
            #a la siguiente generación. Elitismo.

            # Rellenamos el resto de la población mediante reproducción. Utiliza
            while len(new_population) < self.population_size:
                # Seleccionamos dos padres
                p1 = self.tournament_select(scored, k=3)
                p2 = self.tournament_select(scored, k=3)
```

```
                # Cruzamos a los padres para crear hijos, o simplemente los clon
                if random.random() < self.crossover_rate:
                    c1, c2 = self.one_point_crossover_jornada(p1, p2)
                else:
                    c1, c2 = p1[:], p2[:]

                # Mutamos a los hijos y los añadimos a la nueva generación.
                self.mutate(c1)
                new_population.append(c1)
                if len(new_population) < self.population_size:
                    self.mutate(c2)
                    new_population.append(c2)

            # La poblacion pasa a ser la "nueva generacion"
            population = new_population
            # Volvemos a puntuar las jornadas
            scored = self.evaluate(population)

            # Si el mejor de esta generación es mejor que el récord histórico ac
            if scored[0][0] > best_score:
                best_score, best_ind = scored[0]

        # Devolvemos la mejor solucion encontrada despues de todas las generacio
        return best_score, best_ind



# 4. EJECUCION DEL ALGORITMO


# Definimos un objeto de la clase con los parametros que definimos al principio
ga = GeneticAlgorithm(
    population_size=POPULATION_SIZE, # Tamaño de la poblacion
    num_generations=NUM_GENERATIONS, # Numero generaciones del algoritmo
    seed=42, # Establecemos semilla
)

# Aplicamos la funcion evolve para obtener la mejor solucion encontrada y su aud
best_score, best_ind = ga.evolve()

# Mostramos audiencia
print("================================")
print(f"Mejor audiencia total encontrada: {best_score:.2f}")

# Mostramos la mejor solucion encontrada, aplicamos la funcion que definimos ant
print(formatear_jornada(best_ind))
print("================================")
```

```
================================
Mejor audiencia total encontrada: 15.90
Solución de Jornada Óptima:
  Lun 20:00: Equipo 15 (C) vs Equipo 17 (C)
  Sab 20:00: Equipo 4 (B) vs Equipo 10 (B)
  Sab 12:00: Equipo 11 (C) vs Equipo 19 (C)
  Dom 22:00: Equipo 1 (A) vs Equipo 20 (C)
  Sab 22:00: Equipo 3 (A) vs Equipo 9 (B)
  Sab 18:00: Equipo 2 (A) vs Equipo 16 (C)
  Dom 20:00: Equipo 7 (B) vs Equipo 18 (C)
  Dom 18:00: Equipo 8 (B) vs Equipo 13 (C)
  Sab 16:00: Equipo 12 (C) vs Equipo 14 (C)
  Vie 20:00: Equipo 6 (B) vs Equipo 5 (B)
================================
```

Solución mediante fuerza bruta

In [ ]:

```python
import random # Para la semilla de los numeros aleatorios




# Datos del problema(Restricciones)

# Restriccion 2: Audiencias base (Sábado 22h)
# Definimos diccionario
AUDIENCIAS_BASE = {
    ('A', 'A'): 5.0, ('A', 'B'): 4.3, ('A', 'C'): 3.5,
    ('B', 'A'): 4.0, ('B', 'B'): 3.5, ('B', 'C'): 2.0,
    ('C', 'A'): 2.8, ('C', 'B'): 1.7, ('C', 'C'): 1.0,
}

# Restriccion 3: Coeficientes de reducción por horario
# Definimos diccionario
COEFICIENTES_HORARIO = {
    "Viernes 20:00": 0.5, "Lunes 20:00": 0.5,
    "Sabado 12:00": 0.55, "Sabado 16:00": 0.7, "Sabado 18:00": 0.9,
    "Sabado 20:00": 0.9, "Sabado 22:00": 1.0,
    "Domingo 12:00": 0.6, "Domingo 16:00": 0.9, "Domingo 18:00": 0.7,
    "Domingo 20:00": 0.8, "Domingo 22:00": 0.7,
}
# Extraemos solo los nombres de los horarios (claves) del diccionario anterior y
HORARIOS = list(COEFICIENTES_HORARIO.keys()) # Lista de la forma ("Viernes 20:00

# Restriccion 5: Coeficientes de reducción por coincidencia
# Definimos un diccionario
COEFICIENTES_COINCIDENCIA = {
    0: 1.0, 1: 0.75, 2: 0.6, 3: 0.5, 4: 0.4, 5: 0.35, 6: 0.3, 7: 0.25,
}

# Restriccion 2: Categorías de equipos
# Creamos listas que luego utilizaremos para el diccionario de las categorias
EQUIPOS_A = []
for i in range(1, 4):
    EQUIPOS_A.append("Equipo" + str(i))

EQUIPOS_B = []
for i in range(1, 11):
    EQUIPOS_B.append("Equipo" + str(i))
```

```
EQUIPOS_C = []
for i in range(1, 8):
    EQUIPOS_C.append("Equipo" + str(i))

# Unimos las listas las listas
EQUIPOS_TOTALES = EQUIPOS_A + EQUIPOS_B + EQUIPOS_C

#Creamos el diccionario de las categorias con las listas anteriores
CATEGORIA = {}

for equipo in EQUIPOS_A:
    CATEGORIA[equipo] = "A"

for equipo in EQUIPOS_B:
    CATEGORIA[equipo] = "B"

for equipo in EQUIPOS_C:
    CATEGORIA[equipo] = "C"
# De esta forma asocioamos los indices de 1 a 20 a cada categoria para identific


# Funciones "auxiliares" (calculo de audiencia y validacion de jornadas segun la
# Estas funciones las utilizaremos luego en la funcion del algoritmo. Un poco an

# Calculo de audiencias de cada jornada
def calcular_audiencia_jornada(jornada):
    # jornada es una lista de tuplas: (local_str, visitante_str, dia_hora_str) e

    # Inicializamos el conteo como una lista vacia
    conteo_horarios = {}
    # Recorremos la jornada
    for _, _, horario in jornada:
        # Para cada partido calculamos coincidencias y guardamos las coincidenci
        conteo_horarios[horario] = conteo_horarios.get(horario, 0) + 1 # Primero
        # La funcion get no la conocia de antes pero me resulto bastante util

    # Inicializamos la audiencia
    audiencia_total = 0

    # Calculamos audiencia por partido
    # Recorremos la jornada
    for local, visitante, horario in jornada:
        # Para cada partido calculamos lo siguiente:
        cat_local = CATEGORIA[local] # Buscamos la categoría de los equipos del
        cat_visitante = CATEGORIA[visitante]
        base = AUDIENCIAS_BASE[(cat_local, cat_visitante)] # Buscamos la audienc
        mult_h = COEFICIENTES_HORARIO[horario] # Buscamos el coeficiente por hor

        num_coincidencias = conteo_horarios[horario] - 1 # Contamos el numero de
        mult_c = COEFICIENTES_COINCIDENCIA.get(num_coincidencias, COEFICIENTES_C

        audiencia_total += base * mult_h * mult_c # Sumamos la udiencia de los p

    return audiencia_total # Devolvemos la audiencia total de la jornada


# Validacion de cada jornada
def validar_restricciones(jornada):
```

```
    # Verificamos restriccion 4 (todos juegan una vez, no contra sí mismos, todo
    equipos_jugando = set()
    dias_jugados = set()
    # Recorremos la jornada
    for local, visitante, horario_str in jornada:
        # Vemos que no jueguen dos veces o jueguen contra si mismos
        if local in equipos_jugando or visitante in equipos_jugando or local ==
            return False # Equipo repetido o juega consigo mismo
        # Los añadimos a los equipos que estan jugando de local y visitante
        equipos_jugando.add(local)
        equipos_jugando.add(visitante)

        # Comprobamos que todos los dias se jueguen
        dia = horario_str.split(' ')[0] # Nos quedamos solo con el dia que se ju
        # Vamos comrobando y añadiendo los dias si se juega ese dia
        if dia == 'Viernes' or dia == 'Lunes':
            dias_jugados.add(dia)
        elif dia == 'Sabado':
            dias_jugados.add('Sábado')
        elif dia == 'Domingo':
            dias_jugados.add('Domingo')

    # Comprobamos que hemos hecho bien todo y que todos los equipos juegan
    if len(equipos_jugando) != 20:
        return False # No todos los equipos juegan

    # Comprobamos que lo hemos hecho bien y que todos los dias se juega
    if not all(dia in dias_jugados for dia in ['Vie', 'Sábado', 'Domingo', 'Lun'
         return False

    # Devolvemos true para validar
    return True
# Observemos que si no se juegan todos los dias o no todos los equipos juega se



# Algoritmo Fuerza Bruta

def resolver_fuerza_bruta_basica():

    # Inicializamos la audiencia y la jornada para ir guardando nuestros resulta
    mejor_audiencia = 0.0
    mejor_jornada = None


    # Recorremos todos los emparejamientos posibles. 20!/10! como vimos antes en
    for emparejamientos in todas_las_combinaciones_de_emparejamientos():
         # Recorremos todos los posibles horarios. La N que definimos al aplicar
         for horarios_asignados in todas_las_combinaciones_de_horarios():
             # Hacemos una combinacion de las dos(horarios y emparejamientos) pa
             jornada = combinar(emparejamientos, horarios_asignados)
             # Validamos la jornada. Si es correcta nos devuelve un true por lo
             if validar_restricciones(jornada):
                 audiencia = calcular_audiencia_jornada(jornada)
                 # Vemos si es mejor audiencia que la mejor que hemos conseguido
                 if audiencia > mejor_audiencia:
                     mejor_audiencia = audiencia
                     mejor_jornada = jornada
```

```
    # Despues de recorrer todas las posibilidades devolvemos el mejor resultado
    return mejor_audiencia, mejor_jornada



## Como vemos esta ultima parte lo he hecho en forma de pseudocodigo ya que como
## Creo que la idea del codigo se entiende de todas formas. Además el codigo hec
## Para el calculo de todas las posibles combinaciones hariamos bucles anidados
```

Tiempo Abstracto de Ejecución

Nos piden medir la complejidad de ambos algoritmos. Para ello atilizaremos el TAE(Tiempo abstracto de ejecución). Es una técnica para medir el rendimiento de un algoritmo contando las operaciones elementales que realiza, generalmente en el pseudocódigo, independientemente del hardware o lenguaje utitlizados.

Complejidad algoritmo genético

Para calcular el TAE debemos analizar la función principal evolve y como esta interactua con las funciones de apoyo.

Vamos a definir las variables que vamos a emplezar.

P: tamaño de la población, G: número de generaciones, M: número de partidos(matches en inglés), E: número de individuos del proceso de elitismo(los mejores individuos que pasan a la siguiente generación), K: número de padres(número de individuos del conjunto E que elegimos para la reproducción). Tiene un valor constante de 3.

Antes de centrarnos en la función evolve vamos a analizar las funciones de apoyo.

Fitness: recorre M partidos del cromosoma(individuo) para validarlos(tanto a los partidos como al individuo) y luego recorre los M partidos para calcular la audiencia del individuo. Complejidad por tanto de $\mathcal{O}(M)$.

Evaluate: aplica fitness a cada individuo P y luego ordena la población de mayor a menor puntuación. La ordenación sort tiene una complejidad de $\mathcal{O}(P\log(P))$(lo vimos en clase). Entonces la función evaluate tiene una complejidad de $\mathcal{O}(P\log(P)+PM)$.

Mutate: opera sobre los partidos de un individuo para mutarlo. Por tanto, complejidad de $\mathcal{O}(M)$.

Ahora vamos a centrarnos en la función evolve.

1-Elitismo: Se añaden los mejores E individuos de una generación en la siguiente generación. E depende del tamaño de la población y del ratio de elitismo. Como cada individuo tiene M partidos tenemos una complejidad de $\mathcal{O}(EM)$.

2-Reproducción Vamos siguiendo un procedimiento.

Generamos nuevos individuos a partir de los K mejores individuos del conjunto E. Primero que nada elegimos a dos padres en función de este K. Al ser K = 3 constante consideramos que tiene una complejidad de $\mathcal{O}(1)$. Básicamente esta parte no añade complejidad al algoritmo.

Luego hacemos los cruces(crossovers) creandose dos listas combinando partidos de dos padres por lo que tenemos una complejidad de $\mathcal{O}(M)$.

Por último, realizamos un proceso de mutación accediendo a los partidos y modificandose los valores. También tiene una complejidad de $\mathcal{O}(M)$.

Entonces la parte de reproducción tiene una complejidad de $\mathcal{O}((P-E)M)$.

3-Evaluación y ordenación: Básicamente aplicamos la función evaluate. Como comentamos antes tiene una complejidad de $\mathcal{O}(P\log(P)+PM)$.

Para finalizar, aplicamos después del algoritmo genético la función formatear_jornada para mostrar la solución por pantalla. Simplemente es un bucle que recorre los partidos de dicha jornada por lo que tiene una complejidad de $\mathcal{O}(M)$.

Por tanto la complejidad final es $\mathcal{O}(G(EM+(P-E)M+PM+P\log(P))+M)$.

Operando dicha expresión obtenemos:

$\mathcal{O}(G(2PM+P\log(P))+M)$ que es igual a $\mathcal{O}((2GP+1)M+GP\log(P)))$.

Obtenemos así una complejidad de:

$\mathcal{O}(GPM+GP\log(P))$

Complejidad algoritmo fuerza bruta

La complejidad en este caso, una vez que ya hemos calculado todas las posibilidades que tenemos, es bastante simple. Recordemos que las posibilidades con restricciones eran $\frac{20!}{10!}\cdot N$ con N el resultado de aplicar el principio de inclusion-exclusion.

Luego como el codigo es simplemente aplicar la validacion y el calculo de audiencia para cada jornada, que básiamente consiste en aplicar 1 y 2 bucles for a los partidos(que son 10) respectivamente, tenemos entonces una complejidad del orden de $\mathcal{O}(\frac{20!}{10!}\cdot N\cdot 10^2)$. Obviamente el 2 lo simplificamos.

Aquí hay que tener en cuenta que, como no hemos especificado como calcular las posibles combinaciones de horarios y partidos, realmente habría que sumar al orden que hemos calculado el orden de hacer estos calculos. Seguramente lo haría con varios bucles anidados. Aun así el orden del algoritmo de fuerza bruta lo hemos acotado por abajo y comparandolo con el orden de complejidad del algoritmo genetico nos basta para ver la importancia de aplicar esta metodología a este problema. En esencia nos permite intentar resolver un problema que mediante fuerza bruta no podríamos. También es importante ver que no estamos resolviendo el problema en cuanto a

conseguir la mejor jornada posible, si no que estamos consiguiendo la mejor jornada posible con un cierto margen de error que nos permitimos ya que no podemos saber si es la mejor solucion de todas las posible.

En conclusión, lo importante de estudiar el tiempo de ejecución teórico es que nos permite ver de antemano que aproximación a un problema no es válida.